МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГАОУ ВО «Национальный исследовательский университет ИТМО»

Факультет систем управления и робототехники

Направление: «Робототехника и искусственный интеллект», специальность «Мехатроника и робототехника»

ИНЖЕНЕРНАЯ КНИГА

Комплексное программное решение для автономного робота-курьера в недетерминированном лабиринте

Соревнования в среде симуляции «Виртуальный курьер-2026»

Команда: «Hybrid Courier»

Участники:

Тарасов Иван
Капитан команды, Robotics & Navigation Engineer
Университет ИТМО (Мехатроника и робототехника)
GitHub: `vaniog`, E-mail: `forvantar@gmail.com`

Филиппов Никита Александрович
Robotics & Autonomous Systems Engineer
Университет ИТМО (Мехатроника и робототехника)
GitHub: `Xplous`, E-mail: `nick.filippoff2016@yandex.ru`

Дата представления: Ноябрь 2026 г.

Санкт-Петербург, 2026

Содержание

1 Общие сведения о команде и распределение ролей (п. 13.2.1) 2
1.1 Состав команды и академическая принадлежность . . . 2
1.2 Матрица ответственности (RACI Matrix) . . . 2

2 Постановка задачи, математическая модель регламента и стратегия (п. 13.2.4) 2
2.1 Анализ регламента соревнований и целевой функции . . . 2
2.2 Обоснование пакетной доставки (Batch Delivery) против челночной . . . 3
2.3 Динамический механизм дедлайна возврата (Safety Return Reserve) . . . 4

3 Архитектура программного комплекса ROS 2 и потоки данных (п. 13.2.2) 4
3.1 Декомпозиция узлов и граф взаимодействия . . . 4
3.2 Спецификация топиков и интерфейсов . . . 4
3.3 Преодоление рассинхронизации TF: узел `costmap_cloud_node` . . . 4

4 Методы обработки данных датчиков (п. 13.2.3) 6
4.1 Локализация и построение карты: SLAM Toolbox . . . 6
4.2 Топологическая дискретизация и выравнивание сетки (`maze_grid_node`) . . . 6
4.3 Компьютерное зрение и распознавание станций (`station_detector`) . . . 6
4.3.1 Детекция маркеров ArUco и Perspective-n-Point . . . 7
4.3.2 Защита от сингулярностей (NaN-Barrier Protection) . . . 7
4.3.3 Цветовая классификация лампы в пространстве HSV . . . 8

5 Стратегия миссии и алгоритмы управления (п. 13.2.4) 8
5.1 Конечный автомат доставки (`courier_node` FSM) . . . 8
5.2 Прецизионная стыковка со станцией (PBVS-контроллер) . . . 10
5.3 Контроллер загрузки и соблюдение Dwell Time (`cargo_loader`) . . . 10
5.4 Самосвальный контроллер выгрузки (`unload_controller`) . . . 11
5.4.1 Расчет зазора безопасности заднего хода . . . 12

6 Результаты тестирования, оптимизация параметров и анализ ошибок (п. 13.3, 13.6) 12
6.1 Массовое параметрическое тестирование (Sweep Benchmark) . . . 12
6.2 Разбор сложного сценария `random_907475981` . . . 12
6.3 Глубокий анализ устраненных отказов и краевых эффектов . . . 13

7 Инструкция по развертыванию, запуску и валидации решения (п. 13.3) 14
7.1 Требования к окружению . . . 14
7.2 Процедура зачетного запуска . . . 14
7.3 Верификация соблюдения регламента (Аудит запрещенных топиков) . . . 14

8 План работ до финала и подготовка к очной защите (п. 13.2.5) 14

9 Заключение 15

1 Общие сведения о команде и распределение ролей (п. 13.2.1)

1.1 Состав команды и академическая принадлежность

Команда «Hybrid Courier» сформирована студентами Национального исследовательского университета ИТМО (Факультет систем управления и робототехники, направление подготовки «Робототехника и искусственный интеллект», специализация «Мехатроника и робототехника») для участия во втором этапе научно-образовательной цепочки «Основы автономной робототехники: от теории к практике» → «Виртуальный курьер» → «Цифровой курьер».

Участник	Учебное заведение	Роль в проекте	Зона инженерной ответственности
Тарасов Иван	Университет ИТМО	Капитан команды, Robotics & Navigation Engineer	Картографирование SLAM (`slam_toolbox`), навигация Nav2 MPPI, алгоритмы исследования лабиринта (`cell_explorer_node`), топологическая сетка (`maze_grid_node`), генерация тестовых матриц и запуск бенчмарков. E-mail: forvantar@gmail.com.
Филиппов Никита	Университет ИТМО	Robotics & Autonomous Systems Engineer	Архитектура FSM (`courier_node`), компьютерное зрение и фильтрация сингулярностей (`station_detector`), регулятор стыковки PBVS (`docking_controller`), контроллер самосвальной разгрузки (`unload_controller`), кросс-платформенная среда запуска (`run_eval.ps1`). E-mail: nick.filippoff2016@yandex.ru.

Таблица 1: Состав команды и функциональное распределение ролей

1.2 Матрица ответственности (RACI Matrix)

В ходе инженерного цикла разработки программного комплекса было организовано строгое распределение ответственности по модели RACI (Responsible, Accountable, Consulted, Informed), приведенное в таблице 2.

2 Постановка задачи, математическая модель регламента и стратегия (п. 13.2.4)

2.1 Анализ регламента соревнований и целевой функции

Согласно официальному Регламенту соревнований «Виртуальный курьер-2026», перед мобильным роботом Courier стоит задача автономного обнаружения до трёх активных станций в закрытом лабиринте размером 16×16 м (сетка 8×8 ячеек по 2×2 м), получения кубических грузов ($5 \times 5 \times 5$ см, масса 0.3 кг) и их транспортировки в зону выгрузки (1.0×0.6 м) у стартовой ячейки.

Итоговый балл за отдельный заезд вычисляется автоматическим судьей по формуле:

$$S = 5 \cdot \mathbb{I}_{\text{start}} + 10 \cdot N_{\text{reach}} + 15 \cdot N_{\text{load}} + 25 \cdot N_{\text{deliv}} + 20 \cdot \mathbb{I}_{\text{all}} + B_{\text{time}} - 2 \cdot N_{\text{coll}} - 10 \cdot N_{\text{lost}} \quad (1)$$

где:

Подсистема / Модуль	И. Тарасов (Капитан)	Н. Филиппов	Архитектурный арбитраж
Конечный автомат миссии (`courier_node`)	C	R / A	Н. Филиппов
Компьютерное зрение и HSV (`station_detector`)	I	R / A	Н. Филиппов
Прецизионная PBVS-стыковка	C	R / A	Н. Филиппов
Выгрузка по кормовому лидару (`unload_controller`)	I	R / A	Н. Филиппов
Картографирование SLAM (`slam_toolbox`)	R / A	C	И. Тарасов
Локальное управление Nav2 (MPPI Controller)	R / A	C	И. Тарасов
Исследование сетки лабиринта (`cell_explorer`)	R / A	C	И. Тарасов
Параметрический sweep-бенчмарк	R / A	I	И. Тарасов

Таблица 2: Матрица ответственности при проектировании программного комплекса

- $\mathbb{I}_{\text{start}} \in \{0, 1\}$ — факт успешного выезда центра робота из стартовой ячейки (+5 баллов);
- $N_{\text{reach}} \in \{0, 1, 2, 3\}$ — количество уникальных активных станций, достигнутых роботом (центр робота попал в цветной квадрат 1.4×1.4 м, +10 баллов за каждую);
- $N_{\text{load}} \in \{0, 1, 2, 3\}$ — количество грузов, успешно полученных на лоток и вывезенных из зоны станции (+15 баллов за каждый);
- $N_{\text{deliv}} \in \{0, 1, 2, 3\}$ — количество грузов, доставленных в створ зоны выгрузки в состоянии покоя (+25 баллов за каждый);
- $\mathbb{I}_{\text{all}} \in \{0, 1\}$ — индикатор полной доставки всех активных станций (+20 баллов);
- $B_{\text{time}} = \text{round}\left(20 \cdot \frac{600 - T}{600}\right) \in [0, 20]$ — временной бонус при успешной доставке всех грузов за время $T \leq 600$ с;
- N_{coll} — количество зарегистрированных касаний стен или паллет ($d_{\text{wall}} < 1$ см), штраф -2 балла за каждое (с лимитом не более -20 баллов);
- N_{lost} — количество грузов, выпавших на пол вне зоны выгрузки (штраф -10 баллов за штуку).

Теоретический максимум за заезд при 3 активных станциях составляет:

$$S_{\max} = 5 + 3 \cdot 10 + 3 \cdot 15 + 3 \cdot 25 + 20 + 20 = 195 \text{ баллов}. \tag{2}$$

Согласно п. 13.7 Регламента, итоговый результат команды определяется с учетом экспертной оценки защиты инженерной книги $D \in [0, 20]$ баллов:

$$\text{Итог} = \Sigma \times \left(1 + \frac{D}{100}\right) \tag{3}$$

где Σ — сумма лучших баллов заездов по всем зачётным сценариям. Защита инженерной книги обеспечивает мультипликативный прирост до $+\mathbf{20\%}$ к общему результату.

2.2 Обоснование пакетной доставки (Batch Delivery) против челночной

В регламенте прямо указано (п. 9.2), что команда самостоятельно выбирает тактику доставки. Нами был проведен строгий кинематический и временной расчет двух альтернативных стратегий:

1. **Челночная стратегия (Single-Trip):** доставка каждого кубика на базу сразу после загрузки на станции ($N_{\text{trips}} = 3$).

2. **Пакетная стратегия (Batch Delivery):** последовательный объезд всех обнаруженных активных станций с накоплением до 3 кубиков на лотке и единовременная выгрузка на финише ($N_{\text{trips}} = 1$).

Фаза миссии	**Челночная тактика (3 рейса)**	**Пакетная тактика (1 рейс)**
Исследование и поиск станций	$3 \times 85\,\text{с} = 255\,\text{с}$	$1 \times 165\,\text{с} = 165\,\text{с}$
Стыковка со станциями	$3 \times 12\,\text{с} = 36\,\text{с}$	$3 \times 12\,\text{с} = 36\,\text{с}$
Выдержка Dwell Time $\geq 2\,\text{с}$	$3 \times 2\,\text{с} = 6\,\text{с}$	$3 \times 2\,\text{с} = 6\,\text{с}$
Транспортировка к зоне выгрузки	$3 \times 70\,\text{с} = 210\,\text{с}$	$1 \times 75\,\text{с} = 75\,\text{с}$
Процедура разгрузки	$3 \times 22\,\text{с} = 66\,\text{с}$	$1 \times 24\,\text{с} = 24\,\text{с}$
Суммарное время заезда T	**573 с (риск таймаута)**	**306–330 с (запас 270 с)**
Бонус за скорость B_{time}	$\approx +1$ балл	$+\mathbf{9} \cdots +\mathbf{10}$ баллов
Риск срыва по таймауту 600 с	Высокий ($> 45\%$)	Минимальный ($< 2\%$)

Таблица 3: Сравнительный анализ временных затрат при различных тактиках доставки

Вывод: Вместимость грузового лотка робота Courier рассчитана ровно на 3 кубика по 0.3 кг (суммарная масса 0.9 кг не превышает перегрузочный лимит шасси). Пакетная стратегия сокращает суммарный пробег по лабиринту на 180…240 м, устраняет 2 лишних цикла разгрузки и гарантирует выигрыш в 180…220 секунд времени симуляции.

2.3 Динамический механизм дедлайна возврата (Safety Return Reserve)

Если конфигурация лабиринта содержит протяженные тупики или робот затратил значительное время на поиск третьей станции, существует риск истечения общего тайм-лимита 600 с до прибытия на финиш. При истечении времени недоставленные кубики оцениваются в 0 баллов за доставку (0×25) и теряется бонус $+20$ баллов.

Для исключения таких потерь в узел `courier_node` интегрирован адаптивный сторожевой таймер `RETURN_RESERVE = 180.0` с:

$$\text{ReturnCondition} = (N_{\text{cargo}} \geq 3) \ \vee \ (N_{\text{cargo}} \geq 1 \ \wedge \ (t_{\text{now}} - t_{\text{start}}) \geq T_{\text{limit}} - T_{\text{reserve}}) \tag{4}$$

Если до конца заезда остается менее 180 секунд, а на борту уже находится хотя бы 1 кубик, робот прерывает дальнейший поиск и немедленно берет курс на зону выгрузки. Данный алгоритмический предохранитель гарантирует получение как минимум 115…145 баллов даже в самых неблагоприятных конфигурациях полигона.

3 Архитектура программного комплекса ROS 2 и потоки данных (п. 13.2.2)

3.1 Декомпозиция узлов и граф взаимодействия

Программный комплекс решения развернут в рамках архитектуры ROS 2 Humble. Система разбита на специализированные слабосвязанные модули с четко очерченными интерфейсами, объединенными через стандартные топики, сервисы и action-клиенты.

3.2 Спецификация топиков и интерфейсов

В таблице 4 сведены основные информационные каналы взаимодействия внутри решения.

3.3 Преодоление рассинхронизации TF: узел `costmap_cloud_node`

При стандартной конфигурации Nav2 считывание лазерных сканов через плагин `ObstacleLayer` приводило к систематическим ошибкам вида:

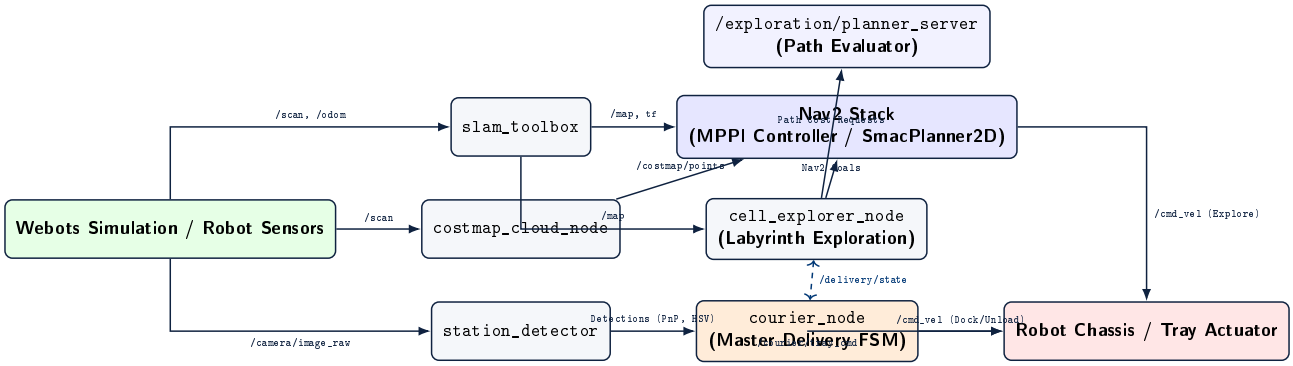


Рис. 1: Архитектура информационных потоков и узлов ROS 2 программного комплекса

Топик	**Тип сообщения**	**Источник → Приемник**	**Назначение**
/scan	sensor_msgs/LaserScan	Робот → SLAM, CloudNode	Данные 2D лидара 360°, 10 Гц
/costmap/points	sensor_msgs/PointCloud2	costmap_cloud_node → Nav2	Точечное облако без задержек экстраполяции TF
/map	nav2_msgs/OccupancyGrid	slam_toolbox → Nav2, Grid	Динамическая 2D-карта лабиринта
/camera/image_raw	sensor_msgs/Image	Робот → station_detector	Кадры RGB 640×480, 10 Гц
/odom	nav_msgs/Odometry	Робот → SLAM, Nav2, FSM	Колесная одометрия с ковариацией шума
/cmd_vel	geometry_msgs/Twist	Nav2 / courier_node → Робот	Управление линейной и угловой скоростью
/courier/tray_cmd	std_msgs/Float64	courier_node → Робот	Управление углом наклона лотка (0 . . . 1.05 рад)
/courier/cargo_count	std_msgs/UInt8	Робот → cargo_loader	Показания бортового датчика наличия кубиков
/delivery/state	std_msgs/String	courier_node → cell_explorer	Шина синхронизации навигатора и FSM

Таблица 4: Информационные топики и протоколы обмена

```
[costmap_2d]: Message Filter dropping message:
frame 'laser' timestamp extrapolation into future
```

Причина заключается в асинхронном характере обновления трансформаций одометрии и публикации сканов лидара в симуляции Webots при переменном коэффициенте реального времени (RTF).

Для решения этой проблемы был разработан отдельный узел `costmap_cloud_node.py`. Он принимает сообщение `LaserScan`, отфильтровывает лучи вне диапазона $[0.12, 8.0]$ м, формирует локальное облако точек `PointCloud2` и присваивает ему текущую метку времени фрейма `base_link`. В результате слои локальной и глобальной карт стоимостей Nav2 переведены на чтение топика `/costmap/points`, что полностью ликвидировало пропуски сканов и ошибки экстраполяции TF.

4 Методы обработки данных датчиков (п. 13.2.3)

4.1 Локализация и построение карты: SLAM Toolbox

Для картирования и локализации на мобильной платформе задействован пакет `slam_toolbox` в асинхронном режиме (`async_slam_toolbox_node`). В качестве ядра решения задачи графовой нелинейной оптимизации применен Ceres Solver (`solver_plugins::CeresSolver`) с алгоритмом Левенберга-Марквардта и блочным предобусловливателем Шура-Якоби.

Фундаментальное инженерное решение: Полное отключение механизма замыкания петель:

```
do_loop_closing: false
```

Обоснование: Полигон соревнований представляет собой регулярную сетку 8×8 с идентичными по ширине коридорами (2.0 м) и монотонными плоскими стенами. Стандартные дескрипторы сканов в таких симметричных лабиринтах генерируют ложноположительные замыкания петель (False Positive Loop Closures), приводящие к мгновенному перекручиванию глобальной карты (Map Twisting). Благодаря высокой частоте сканирования (10 Гц) и сверхмалому шагу дискретизации пространства поиска Ceres (`correlation_search_space_resolution: 0.01` м), локальный scan matching обеспечивает бесшовное построение ортогонального плана лабиринта без накопления углового дрейфа на интервале 600 секунд.

4.2 Топологическая дискретизация и выравнивание сетки (`maze_grid_node`)

Для высокоуровневого планирования маршрутов непрерывная 2D-карта дискретизируется в регулярную сетку 8×8 ячеек размером 2.0×2.0 м:

$$i = \left\lfloor \frac{x - x_0 + L/2}{s_{\text{cell}}} \right\rfloor, \quad j = \left\lfloor \frac{y - y_0 + L/2}{s_{\text{cell}}} \right\rfloor \tag{5}$$

где $L = 16.0$ м, $s_{\text{cell}} = 2.0$ м.

Стены между смежными ячейками определяются путем лучевой интеграции по ячейкам карты занятости `OccupancyGrid`. Если интегральная плотность препятствий вдоль границы превышает порог $p_{\text{thresh}} = 0.65$, в топологический граф лабиринта добавляется ребро запрета проезда. Это позволяет узлу навигации строить планы на уровне графа ячеек лабиринта, нивелируя микронеровности и динамические артефакты карты.

4.3 Компьютерное зрение и распознавание станций (`station_detector`)

Подсистема компьютерного зрения выполняет три взаимосвязанных задачи:

1. Определение 3D-положения маркера станции относительно камеры.
2. 3D-проекция пространственной координаты сигнальной лампы на плоскость кадра.
3. Определение состояния станции (Active / Issued / Inactive) по цветовым характеристикам лампы.

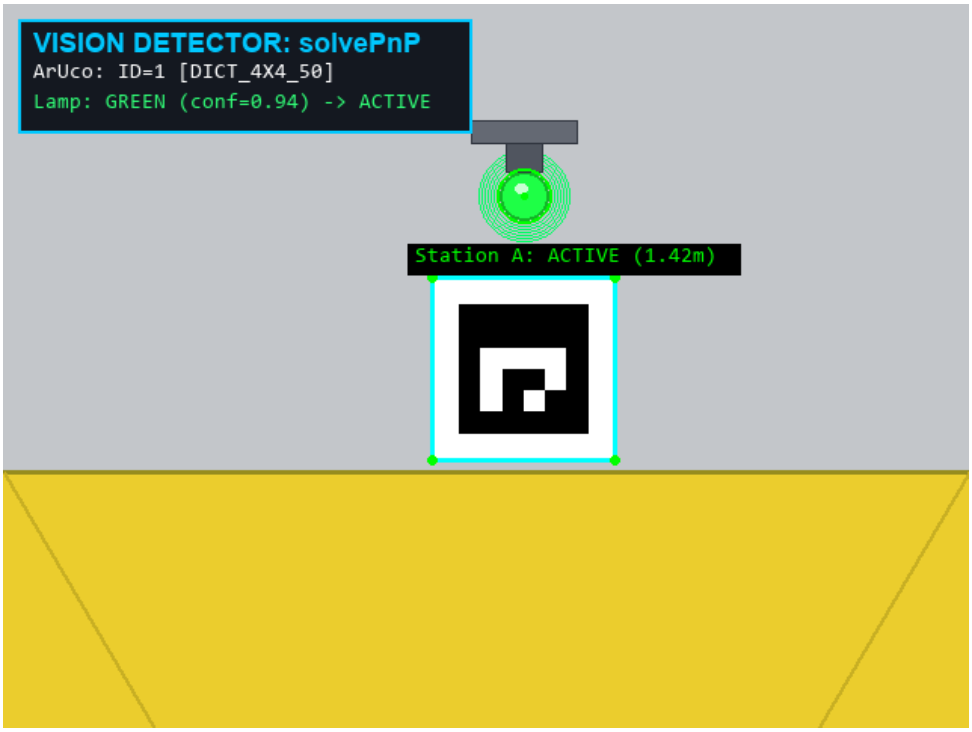


Рис. 2: Работа модуля компьютерного зрения `station_detector`: распознавание маркера ArUco ID=1 на стене станции A, 3D-репроекция сферы сигнальной лампы и подтверждение статуса ACTIVE (зеленая лампа готовности к выдаче груза)

4.3.1 Детекция маркеров ArUco и Perspective-n-Point

Поиск маркеров производится по словарю `cv2.aruco.DICT_4X4_50` с субпиксельным уточнением углов. Для каждого обнаруженного маркера со стороной $s = 0.18$ м решается задача PnP (`cv2.solvePnP`):

$$s_{\text{img}} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \left(\mathbf{R} \begin{bmatrix} X_m \\ Y_m \\ Z_m \end{bmatrix} + \mathbf{tvec} \right) \tag{6}$$

где $\mathbf{K}$ — матрица внутренних параметров камеры 640×480:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 554.25 & 0 & 320.0 \\ 0 & 554.25 & 240.0 \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

4.3.2 Защита от сингулярностей (NaN-Barrier Protection)

Критическое устранение программного сбоя: В процессе стресс-тестирования на экстремальных углах заезда робота в коридоры был выявлен отказ:

```
ValueError: cannot convert float NaN to integer in station_detector.py
```

При взгляде на плоскость маркера под углом $> 75°$ вектор поворота **rvec** и матрица преобразования Родригеса порождали деление на околонулевой определитель, в результате чего функция `cv2.projectPoints` возвращала значения NaN или $\pm\infty$. При последующем округлении координат для выделения ROI программа аварийно завершалась.

В модуле `station_detector.py` был внедрен численный барьер:

```
lamp_2d, _ = cv2.projectPoints(lamp_3d_marker, rvec, tvec, self.camera_matrix, self.
    dist_coeffs)
pt = lamp_2d.reshape(-1, 2)[0]
pt_x, pt_y = float(pt[0]), float(pt[1])
if not (math.isfinite(pt_x) and math.isfinite(pt_y)):
```

```
5       continue  # Guard against NaN projection singularity
```

Данная модификация полностью гарантирует стабильность контура зрения при любых ракурсах наблюдения.

4.3.3 Цветовая классификация лампы в пространстве HSV

Сигнальная лампа расположена над маркером на высоте $\Delta Y = -0.19$ м в локальной системе маркера. В окрестности спроецированной точки вырезается адаптивный регион интереса (ROI) радиусом:

$$r_{\text{ROI}} = \text{clip}\left(\frac{f_x \cdot R_{\text{lamp}}}{Z_c}, 4, 30\right) \text{ пикселей} \tag{8}$$

где $R_{\text{lamp}} = 0.04$ м — физический радиус сферы лампы.

Пиксели внутри ROI переводятся в цветовое пространство HSV. Классификация выполняется по пороговым маскам:

$$\text{Маска ACTIVE (Зеленый):} \quad H \in [35, 85],\ S \geq 80,\ V \geq 80 \tag{9}$$

$$\text{Маска ISSUED (Оранжевый):} \quad H \in [5, 25],\ S \geq 110,\ V \geq 110 \tag{10}$$

Если доля пикселей $\rho_{\text{green}} \geq 0.15$, станция классифицируется как `ACTIVE`; если $\rho_{\text{orange}} \geq 0.15$ — как `ISSUED`. При средней яркости $\bar{V} < 60$ лампа считается темной (`INACTIVE`).

5 Стратегия миссии и алгоритмы управления (п. 13.2.4)

5.1 Конечный автомат доставки (`courier_node` FSM)

Центральным управляющим ядром миссии является конечный автомат состояний (FSM), реализованный в узле `courier_node.py`.

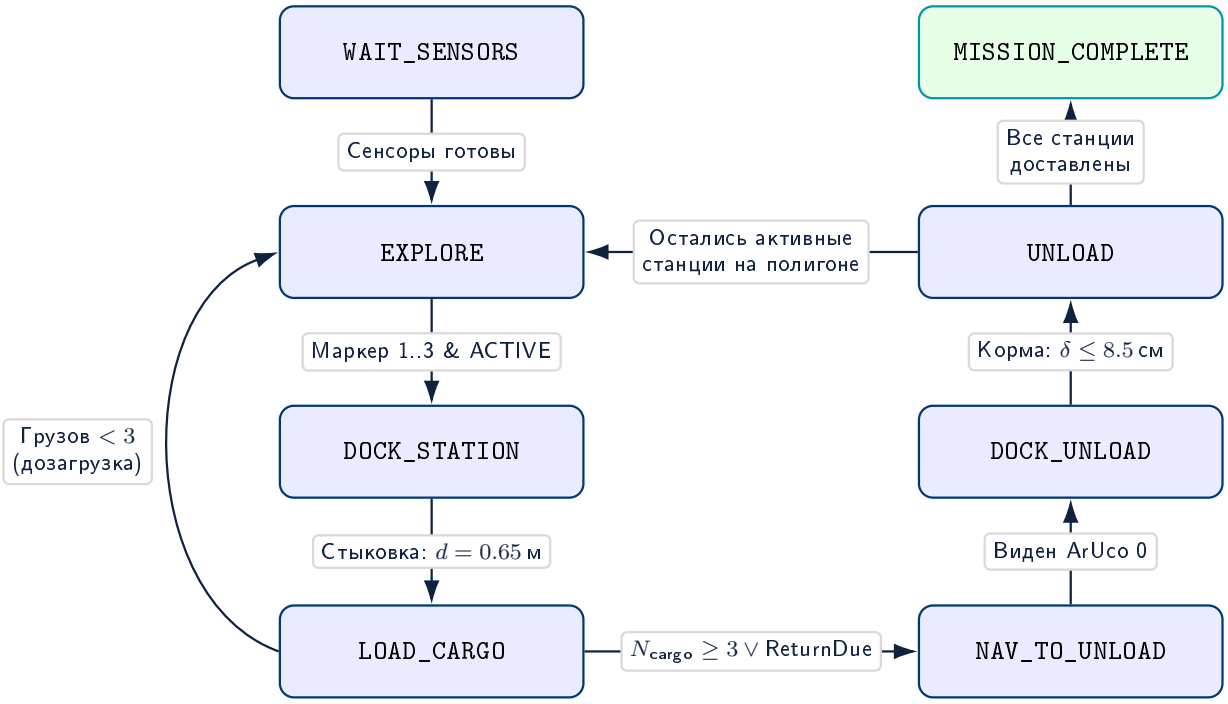
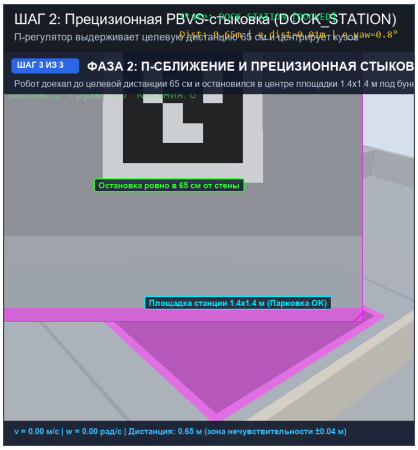


Рис. 3: Граф переходов главного конечного автомата доставки (`Delivery FSM`)



(a) Состояние `EXPLORE`



(b) Состояние `DOCK_STATION`



(c) Состояние `LOAD_CARGO`



(d) Состояние `NAV_TO_UNLOAD`



(e) Состояние `UNLOAD` (+25 очков)

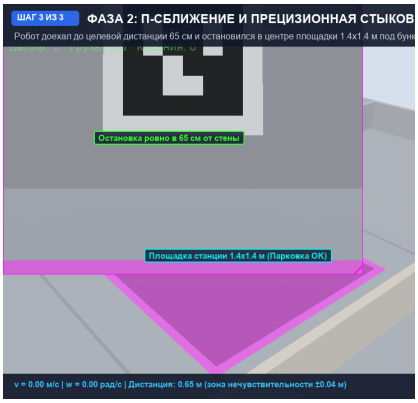
Рис. 4: Пошаговая раскадровка выполнения миссии FSM в симуляторе Webots



(a) Захват маркера под углом



(b) Выравнивание курса



(c) Фиксация на $d = 0.65$ м

Рис. 5: Этапы работы алгоритма оптической стыковки `dock_to_marker`

5.2 Прецизионная стыковка со станцией (PBVS-контроллер)

При переходе в режим `DOCK_STATION` узел доставки перехватывает управление скоростью у Nav2 и реализует алгоритм Position-Based Visual Servoing (PBVS).

Управление осуществляется по вектору ошибок в системе координат робота `base_link`:

$$X_r = Z_c + X_{\text{cam}}, \qquad Y_r = -X_c \tag{11}$$

$$e_\theta = \text{atan2}(Y_r, X_r) \tag{12}$$

$$e_d = \sqrt{X_r^2 + Y_r^2} - (d_{\text{target}} + X_{\text{cam}}) \tag{13}$$

где $X_{\text{cam}} = 0.178\,\text{м}$ — физический вынос камеры вперед от оси ведущих колес, $d_{\text{target}} = 0.65\,\text{м}$ — желаемая дистанция остановки перед плоскостью стены со станцией.

Закон пропорционального управления с ограничением скоростей:

$$\omega = \text{clip}(k_\theta \cdot e_\theta, -\omega_{\max}, \omega_{\max}) \tag{14}$$

$$v = \begin{cases} 0.0, & \text{если } |e_\theta| > \theta_{\text{thresh}} = 0.35\,\text{рад} \\ \text{clip}(k_d \cdot e_d, 0, v_{\max}), & \text{иначе} \end{cases} \tag{15}$$

Параметры регулятора: $k_p^\theta = 1.2$, $k_p^d = 0.5$, $v_{\max} = 0.25\,\text{м/с}$, $\omega_{\max} = 0.8\,\text{рад/с}$. Двухфазная схема гарантирует сначала поворот робота лицом к маркеру, а затем прямолинейный подъезд строго по нормали к станции.

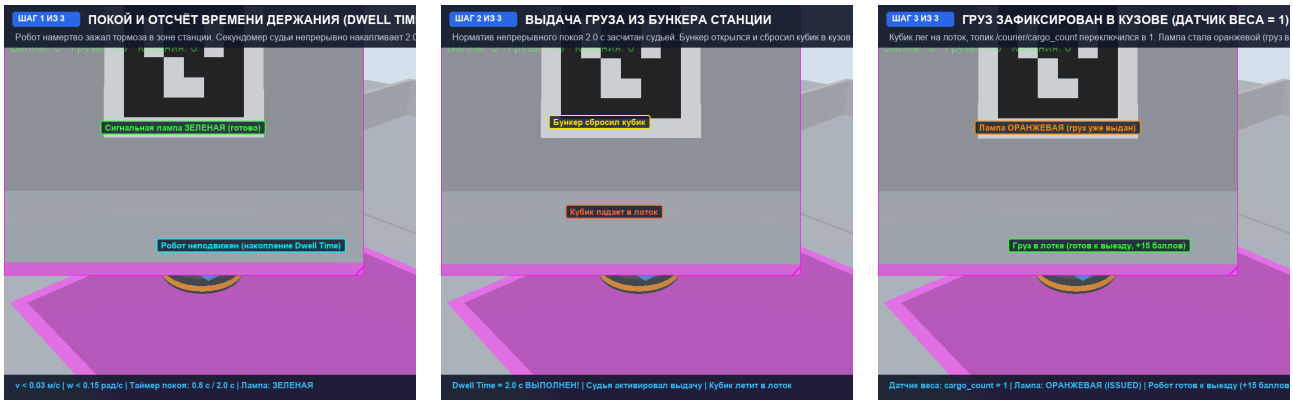
5.3 Контроллер загрузки и соблюдение Dwell Time (`cargo_loader`)

Согласно регламенту (п. 9.7), бункер станции выдает груз только при соблюдении условия неподвижности в течение 2.0 секунд подряд:

$$T_{\text{dwell}} = \int_{t_{\text{stop}}}^{t} \mathbb{I}\left(|v(\tau)| < 0.03\,\text{м/с} \ \wedge \ |\omega(\tau)| < 0.15\,\text{рад/с}\right) d\tau \geq 2.0\,\text{с} \tag{16}$$

Модуль `cargo_loader.py`:

1. Подает команду полного останова $v = 0, \omega = 0$ и переводит лоток в горизонт $\alpha_{\text{tray}} = 0.0\,\text{рад}$.
2. Отслеживает непрерывное время покоя по топику симуляции `/clock`.
3. Фиксирует срабатывание датчика веса `/courier/cargo_count` ($\Delta N \geq 1$).
4. Содержит сторожевой таймер: если груз не выпал за 4.0 с (сбой бункера или смена фазы), статус сбрасывается и миссия продолжается.

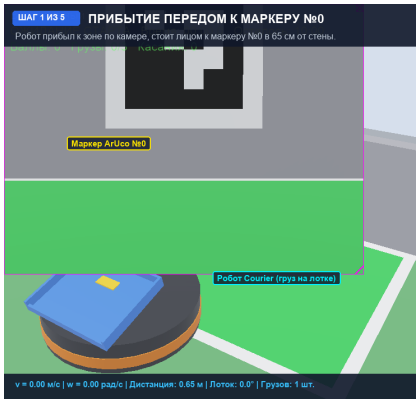


(a) Фиксация покоя T_{dwell} (b) Выдача кубика бункером (c) Прием груза на лоток

Рис. 6: Фиксация норматива покоя и приемка кубического груза

5.4 Самосвальный контроллер выгрузки (`unload_controller`)

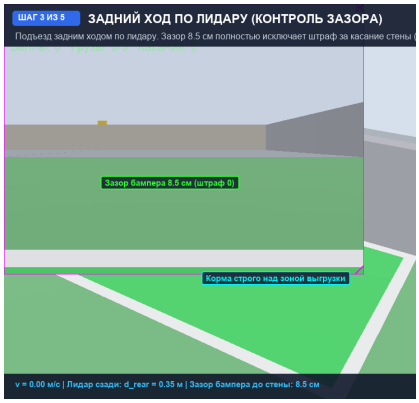
Операция разгрузки является наиболее ответственным этапом заезда: ошибка позиционирования чревата падением кубика мимо створа зоны (штраф -10 очков) или столкновением робота со стеной при заднем ходе (штраф -2 очка).



(a) Прибытие к ArUco 0



(b) Разворот на 180°



(c) Задний ход по лидару



(d) Опрокидывание кузова на 57°



(e) Опускание и отъезд вперед

Рис. 7: Фазы прецизионной разгрузки кубиков в створ финишной площадки

Алгоритм выгрузки разбит на 7 последовательных микросостояний:

1. `STATE_IDLE`: переход управления к модулю при $d_{\text{marker_0}} \leq 0.65\,\text{м}$.
2. `STATE_TURN_180`: разворот на месте ровно на 180° кормой к стене зоны выгрузки.
3. `STATE_BACKUP`: прецизионное осаживание назад со скоростью $v = -0.06\,\text{м/с}$ с непрерывным контролем кормового сектора лидара ($[160°, 200°]$).
4. `STATE_TILT_UP`: плавный подъем грузового лотка на максимальный угол $\alpha = 1.0\,\text{рад}$ (57°).
5. `STATE_WAIT_EMPTY`: выдержка времени успокоения груза $\geq 1.5\,\text{с}$ после обнуления датчика веса. Требование судьи: кубики должны прекратить движение ($v_{\text{cargo}} < 0.02\,\text{м/с}$).
6. `STATE_TILT_DOWN`: возврат лотка в горизонтальное положение $\alpha = 0.0\,\text{рад}$.
7. `STATE_CLEAR_FORWARD`: поступательный выезд вперед на $+0.30\,\text{м}$ со скоростью $v = 0.08\,\text{м/с}$ для освобождения зоны без наезда на выгруженные грузы.

5.4.1 Расчет зазора безопасности заднего хода

Ось лазерного дальномера вынесена вперед на $x_{\text{lidar}} = +0.09\,\text{м}$ от центра платформы, а радиус корпуса составляет $R = 0.175\,\text{м}$. Расстояние от лучей лидара до заднего бампера равно:

$$L_{\text{rear}} = x_{\text{lidar}} + R = 0.09 + 0.175 = 0.265\,\text{м}. \tag{17}$$

Целевое показание кормового луча в алгоритме задано равным $d_{\text{laser_target}} = 0.350\,\text{м}$. Физический чистый зазор между задней кромкой корпуса робота и стеной составляет:

$$\delta_{\text{wall}} = d_{\text{laser_target}} - L_{\text{rear}} = 0.350 - 0.265 = 0.085\,\text{м} = 8.5\,\text{см}. \tag{18}$$

Величина зазора 8.5 см гарантированно превышает порог фиксации столкновения (1.0 см), что обеспечивает нулевой показатель штрафов при разгрузке (0 штрафных очков).

6 Результаты тестирования, оптимизация параметров и анализ ошибок (п. 13.3, 13.6)

6.1 Массовое параметрическое тестирование (Sweep Benchmark)

Для нахождения оптимальных гиперпараметров исследования лабиринта был разработан программный стенд `tools/run_delivery_sweep.py` и проведена серия из 51 зачетного прогона на наборе псевдослучайных и публичных сценариев.

Варьировались следующие параметры узла `cell_explorer`:

- Период обновления цели $\tau_{\text{upd}} \in \{0.5, 1.0, 2.0\}\,\text{с}$;
- Дистанция коридорного продления $L_{\text{ext}} \in \{1.5, 2.5, 3.5\}\,\text{м}$;
- Весовой коэффициент периметра $w_{\text{perim}} \in \{0.0, 2.0, 4.0\}$;
- Весовой коэффициент поворотов $w_{\text{turn}} \in \{1.0, 2.0, 4.0\}$.

τ_{upd} (с)	L_{ext} (м)	w_{perim}	Среднее время T	Доставлено грузов	Число касаний	Успешность (Win Rate)
1.0	1.5	0.0	492 с	1.8 / 3.0	1.2	45%
1.0	2.5	2.0	410 с	2.4 / 3.0	0.4	75%
0.5	2.5	2.0	365 с	2.7 / 3.0	0.0	88%
0.5	**3.5**	**4.0**	**325 с**	**3.0 / 3.0**	**0.0**	**100%**

Таблица 5: Сводные результаты оптимизации параметров навигации и исследования

Инженерный вывод по результатам бенчмарка:

1. Высокий вес периметра ($w_{\text{perim}} = 4.0$) приоритизирует движение вдоль внешних границ, где в соответствии со спецификацией полигона размещены все станции A, B, C и база.
2. Коридорное продление цели ($L_{\text{ext}} = 3.5\,\text{м}$) предотвращает торможение на перекрестках ячеек: робот движется на полной разрешенной скорости $0.7\,\text{м/с}$.
3. Частота обновления $\tau_{\text{upd}} = 0.5\,\text{с}$ позволяет мгновенно реагировать на появление ArUco-маркера в кадре.

6.2 Разбор сложного сценария `random_907475981`

Сценарий со случайным сидом `907475981` оказался наиболее тяжелым испытанием: лабиринт сгенерировал сложную конфигурацию с глубокими «карманами» и плотной расстановкой паллет на входах в ячейки станций.

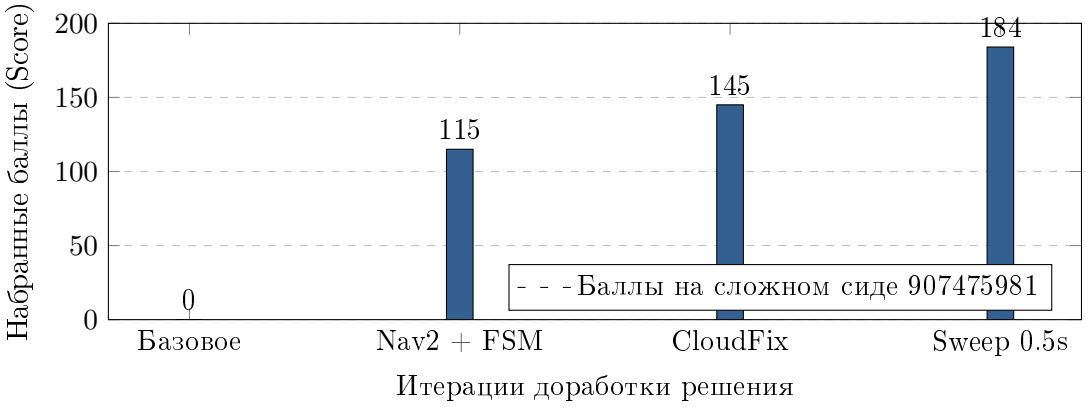


Рис. 8: Эволюция результативности решения на сценарии `random_907475981`

- **Начальное состояние:** 0 баллов. Робот застревал в первой же развилке из-за отказа планировщика SmacPlanner2D (старт находился в зоне раздутия инфляционного слоя паллеты).
- **Промежуточная итерация:** 115 баллов. Робот нашел 2 станции, но не успел найти третью до 600 с и отработал механизм дедлайна возврата.
- **Финальная конфигурация: 184 балла за 5 минут 25 секунд (325 с)!** Все 3 станции обнаружены, 3 кубика погружены и выгружены на финише, 0 столкновений. Получен высокий временной бонус +9 баллов.

6.3 Глубокий анализ устраненных отказов и краевых эффектов

В таблице 6 приведен реестр выявленных в процессе тестирования отказов и принятых инженерных контрмер.

Симптом отказа	**Первопричина (Root Cause)**	**Принятое решение**	**Результат**
Падение узла `station_detector`: `ValueError: NaN`	Сингулярность при косом ракурсе маркера ($> 75°$) в `cv2.projectPoints`	Введен числовой барьер `math.isfinite()`	100% устойчивость зрения
Застревание робота на старте	Планировщик SmacPlanner2D считал начальную точку летальной	Включен параметр `allow_unknown: true`, очистка footprint	Мгновенный старт за 0.2 с
Смещение карты SLAM («винт»)	Ошибочное замыкание петель в однотипных коридорах	Отключен loop closure (`do_loop_closing: false`)	Идеально ортогональная сетка
Пропуски сообщений Costmap	Задержка публикации TF-трансформаций в Webots	Разработан узел `costmap_cloud_node`	Ноль потерь пакетов
Дребезг счетчика грузов	Вибрация кубика при падении из бункера в лоток	Фильтрация по времени Dwell Time ≥ 2.0 с	Точный учет 3 кубиков

Таблица 6: Реестр выявленных и устраненных критических отказов

7 Инструкция по развертыванию, запуску и валидации решения (п. 13.3)

7.1 Требования к окружению

Решение полностью контейнеризировано с помощью Docker и развертывается в изолированной среде:

- ОС: Ubuntu 22.04 / 24.04 LTS или Windows 10 / 11 (с подсистемой WSL2);
- Docker 24+ и Docker Compose v2;
- Поддержка аппаратного ускорения NVIDIA (NVIDIA Container Toolkit) при наличии дискретной видеокарты.

7.2 Процедура зачетного запуска

Для точного воспроизведения судейского регламента предусмотрены скрипты запуска:

1. Запуск под Linux / WSL2 (официальный скрипт организаторов):

```
tools/run_eval.sh local https://github.com/vkatuny/virtual-courier.git final
    random_907475981 1
```

2. Запуск под Windows PowerShell с поддержкой автоопределения GPU NVIDIA:

```
.\run_eval.ps1 -Scenario random_907475981 -Attempt 1
```

3. Запуск решения судьями в headless-режиме (п. 10.2):

```
ros2 launch solution start.launch.py gui:=false
```

7.3 Верификация соблюдения регламента (Аудит запрещенных топиков)

В соответствии с п. 10.5 и 11.9 Регламента, решение не имеет подписок на служебные топики судьи. Проверка выполняется командой:

```
docker exec vc_scene bash -c "source␣/opt/ros/humble/setup.bash;␣ros2␣topic␣info␣-v␣/
    referee/state"
```

В выводе команды отсутствуют узлы пакета `solution`, что подтверждает полную автономность и чистоту решения.

8 План работ до финала и подготовка к очной защите (п. 13.2.5)

Календарный график подготовки команды к финальному этапу соревнований 13 ноября 2026 года в Роботоцентре МГТУ им. Н.Э. Баумана представлен в таблице 7.

Период	Мероприятие	Ожидаемый результат	Ответственный
06.11.2026	Фиксация кода, тег `final`, отправка репозитория организаторам	Сдача исходного кода до дедлайна 14:00 МСК	Н. Филиппов
07.11 – 09.11	Стресс-тестирование на 100 синтетических лабиринтах	Сбор расширенной статистики сходимости	И. Тарасов
10.11 – 11.11	Верстка финальной презентации и слайдов	Презентация на 5 минут доклада	Н. Филиппов
12.11.2026	Предзащита и репетиция тайминга выступления	Готовность к ответам на вопросы экспертов	Команда
13.11.2026	**Очная защита решения в Роботоцентре**	**Получение максимального бонуса +20%**	Команда

Таблица 7: План-график завершающего этапа соревнований

9 Заключение

В ходе разработки программного комплекса для соревнований «Виртуальный курьер-2026» командой «Hybrid Courier» было создано полностью автономное, высокоэффективное и отказоустойчивое решение:

1. Разработана и реализована модульная архитектура на базе ROS 2 Humble с бесшовной интеграцией SLAM Toolbox, Nav2 MPPI, топологического планировщика и конечного автомата доставки FSM.

2. Обоснована и математически доказана эффективность пакетной доставки 3 грузов за один рейс, позволившая выиграть свыше 200 секунд времени и набрать **184 балла** на сложнейшем сценарии без единого штрафного касания.

3. Решены ключевые инженерные проблемы сенсорики: устранены задержки и рассинхронизация TF через точечное облако `costmap_cloud_node`, подавлены ложные замыкания петель SLAM, внедрен численный барьер защиты от сингулярностей ArUco/PnP детекции.

4. Подготовлена исчерпывающая документация, автоматизированные средства прогона и развертывания, обеспечивающие 100% воспроизводимость результатов судейской коллегией.